Task 4 — Shopping Cart & Checkout Summary

Overview

Build a shopping cart that persists per user/session, merges duplicate line items, and recalculates totals reactively. This task exercises quantity math, server-side total computation, and reactive UI state.

User Story

> ***As a*** *shopper,* ***I want*** *to add products to a cart and see a live total,* ***so that*** *I can review my order before checking out.*

Your Story Point Estimation (To Be Completed by Learner)

Estimate the effort for this user story using a Fibonacci scale (1, 2, 3, 5, 8, 13, ...). Consider complexity, effort, risk, and uncertainty, then justify your reasoning.

Factor	Your Assessment
Complexity *(how hard is the logic?)*	*(fill in)*
Effort *(how much work / how many parts?)*	*(fill in)*
Uncertainty / Risk *(unknowns, dependencies?)*	*(fill in)*
Your Estimate (points)	*(fill in)*
Rationale	*(explain how you arrived at your estimate)*

Acceptance Criteria

- Adding a product already in the cart **increments its quantity** rather than creating a duplicate line.
- The cart total updates automatically when items or quantities change.
- Removing an item, or setting its quantity to zero, deletes it from the cart.
- Quantities must be positive integers; invalid quantities are rejected.
- Line total = unit price × quantity; grand total = sum of all line totals.
- An empty cart returns a total of zero and an empty-cart state.

Data Model

Entity	Field	Type	Notes
Cart	`id` / `ownerId`	reference	One active cart per user/session
CartItem	`productId`	reference	Unique per cart (merge on add)
CartItem	`quantity`	integer	≥ 1
CartItem	`unitPrice`	decimal	Snapshot at add time

API Contract (Framework Agnostic)

Method	Endpoint	Purpose	Success	Errors
`GET`	`/cart`	Fetch cart with computed totals	`200`	`401`
`POST`	`/cart/items`	Add item (merge if exists)	`201` / `200`	`400`
`PUT`	`/cart/items/:id`	Update quantity	`200`	`400`, `404`
`DELETE`	`/cart/items/:id`	Remove item	`204`	`404`

`GET /cart` success `200`:

```
{
  "items": [ { "productId": "...", "name": "...", "unitPrice": 9.99, "quantity": 2,
"lineTotal": 19.98 } ],
  "subtotal": 19.98,
  "grandTotal": 19.98
}
```

Back-End Requirements

- Merge duplicate products into a single line by incrementing quantity.
- Compute subtotal, per-line totals, and grand total **server-side** (source of truth).
- Persist the cart per user/session so it survives page reloads.
- Treat `quantity = 0` (or negative) as a removal / validation error consistently.
- Use precise decimal handling to avoid floating-point rounding errors on money.

Front-End Requirements

- Cart view listing items, unit price, quantity, and line totals.
- Quantity stepper and remove button that recalculate totals instantly.
- A summary panel showing subtotal and grand total.
- Empty-cart state with a call-to-action to browse products.
- Disable checkout when the cart is empty.

Non-Functional

- Money values displayed with correct currency formatting and 2 decimals.
- Rapid quantity changes should not produce race conditions or duplicate lines.

Edge Cases to Handle

- Adding the same product multiple times in quick succession.
- Setting quantity to 0 or a negative/non-integer value.

- Removing the last item (cart becomes empty, total = 0).
- Product price changes after it was added (define snapshot vs. live behavior).

Unit Test Scenarios (Given / When / Then)

#	Given	When	Then
1	A product already in the cart	Add the same product	Quantity increments; no duplicate line
2	Unit price 9.99, quantity 3	Totals computed	Line total = 29.97; grand total sums lines
3	An item in the cart	Set quantity to 0	Item is removed from the cart
4	An empty cart	Get cart	Grand total = 0, items empty
5	Quantity set to -1 or 1.5	Update is called	Validation rejects with `400`

Deliverables

- Back-end cart endpoints with merge logic and server-side totals.
- Front-end cart UI with steppers, removal, and live summary.
- Unit tests covering all scenarios above.
- Short README: how to run the app and tests, plus your completed estimation table.